



Statistical Analysis of Line Coding for Digital Communication



Students Information:

- 1- 91240202 باسل شريف حميد مصطفى
- 2- 91240651 محمد حمدي فتحي
- 3- 91240645 محمد جمال جمعه السيد
- 4- 91240586 كيرلس سمير خليل

Academic Year: 2025/2026

Course: Communication, Digital Communication Part

Supervised By: Dr. Mohamed Nafie

1. Introduction

In modern communication systems, the traditional paradigm of hardwired analog signal processing is being superseded by the Software Defined Radio (SDR) approach. This shift represents a fundamental evolution in how we conceive, generate, and process signals. By bridging the gap between digital data and physical waveforms through software, engineers can dynamically modify modulation and coding schemes without the necessity of physical hardware redesigns.

The primary objective of this project is to explore these SDR principles by implementing a software-based transmitter that interfaces with the digital domain. Rather than relying on analog circuitry to define signal characteristics, we utilize software logic to generate and analyze specific line coding schemes: **Unipolar**, **Polar NRZ**, and **Return to Zero (RZ)**.

These line codes are essential for converting raw binary data into voltage waveforms suitable for transmission. However, beyond mere generation, this project investigates the statistical properties of these waveforms. In a practical communication system, transmission is subject to randomness—both in the data content and the timing of the signal. Consequently, we treat the output of our software transmitter as a random process and evaluate it through an ensemble analysis.

The scope of this report encompasses the following core investigations:

- ◆ **Waveform Synthesis:** Developing robust algorithms to create an ensemble of 500 waveforms for each specified line code.
- ◆ **Statistical Analysis:** Computing the ensemble mean and autocorrelation to understand the underlying signal behavior.
- ◆ **Property Verification:** Evaluating the processes for Wide-Sense Stationarity and Ergodicity to determine if time-averaged results from a single realization are representative of the entire ensemble.
- ◆ **Spectral Analysis:** Estimating the bandwidth of these schemes using Fast Fourier Transform (FFT) techniques to assess spectral efficiency.

2. Data Generation

The foundation of the simulation is the creation of a stochastic ensemble. Rather than simulating a single instance, we utilize a Monte Carlo-style approach, generating a large matrix of independent realizations.

2.1 Stochastic Initialization

We generate 500 independent bitstreams. This is critical because a single waveform is insufficient to describe the statistics of the entire process. By utilizing an ensemble, we ensure that our calculated averages converge toward the theoretical properties of the signal.

2.2 Ensemble Matrix Structure:

The ensemble is stored as a 2D matrix where each row represents a unique realization of the transmitter's output. This structure is essential for the later stages of the report, as it allows us to perform column-wise operations (to calculate the ensemble mean) and row-wise operations (to analyze autocorrelation properties).

2.3 Simulation Consistency:

By defining the parameters (num_bits, num_waveforms, samples_per_bit) at the start, we ensure that the entire system—from time-domain generation to frequency-domain FFT analysis—remains synchronized and consistent across all statistical investigations.

3. Creation of unipolar ensemble

```
%% Simulation Setup
clear; clc; close all;
% Parameters
num_bits = 100;
num_waveforms = 500;
samples_per_bit = 7;
% Ensemble Initialization
% 500 waveforms, each with 100 bits * samples_per_bit
ensemble = zeros(num_waveforms, num_bits * samples_per_bit);
for i = 1:num_waveforms
    % Generate 100 random bits for each waveform
    Data = randi([0 1], 1, num_bits);
    % Store the resulting waveform in the ensemble
    ensemble(i, :) = Tx_out; end
```

Code 1: Data Generation

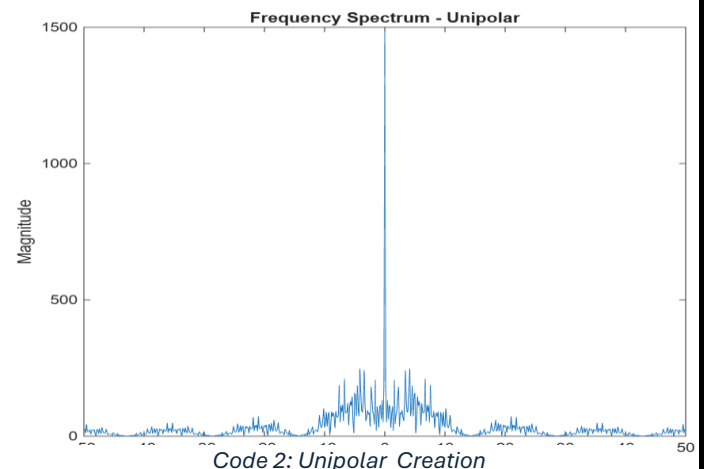
3. Creation of unipolar ensemble

3.1 Methodology

In the Unipolar method, the ensemble is created by mapping bits directly to voltage levels 0 and A. Our code utilizes $T_x = \text{Data} * A$ to perform this mapping. The repmat function then creates the pulse width by repeating each bit value samples_per_bit times, and reshape flattens the matrix into a single waveform vector. This process is repeated 500 times to populate the rows of the ensemble matrix.

```
% Unipolar (0 → 0, 1 → A)
Tx = Data * A;
Tx2 = repmat(Tx,
    samples_per_bit, 1);
Tx_out = reshape(Tx2, 1, []);
```

Code 3: Unipolar Creation



Code 2: Unipolar Creation

4. Creation of Polar NRZ ensemble

4.1 Methodology:

The ensemble for Polar NRZ is created using a symmetric bipolar mapping. Your code implements this via $((2 * \text{Data}) - 1) * A$, which effectively transforms the binary input range $\{0, 1\}$ to $\{-1, 1\}$ before scaling by amplitude A . Similar to the Unipolar method, `repmat` and `reshape` are applied to expand each bit into the required sample count, forming the continuous waveform. This ensures that the generated ensemble contains only voltage levels $\{-A, +A\}$.

```
% Polar NRZ (0 → -A, 1 → +A)
Tx = ((2 * Data) - 1) * A;
Tx2 = repmat(Tx, samples_per_bit, 1);
Tx_out = reshape(Tx2, 1, []);
```

Code 4: NRZ Creation

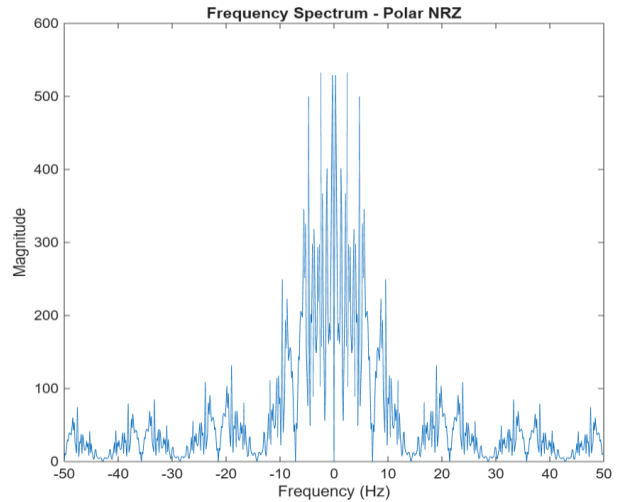


Figure 1: Frequency of NRZ

5. Creation of RZ ensemble

5.1 Methodology

The RZ ensemble creation differs because it requires time-domain partitioning within the bit period. Your implementation uses a for loop to iterate through each bit. The mapping logic assigns the voltage (+A) to the first half of the samples (`start_idx` to `start_idx+3`) and explicitly forces the voltage to 0 for the second half (`start_idx+4` to `start_idx+6`). By filling these indices directly, your code constructs a non-uniform pulse shape that returns to the baseline, which is stored row-by-row into the ensemble matrix.

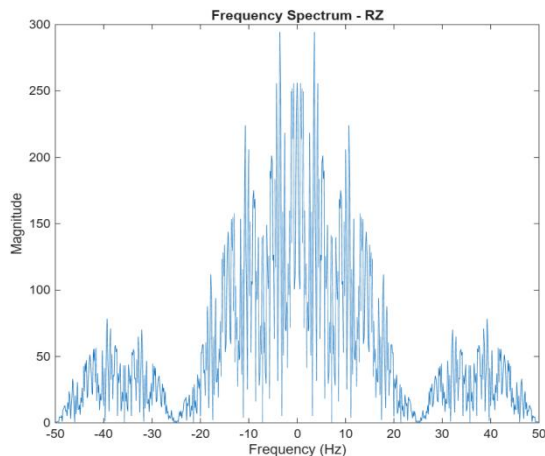


Figure 2: Frequency of RZ

```
% RZ (Return to Zero)
Tx = ((2 * Data) - 1) * A;
Tx_out = zeros(1, num_bits * samples_per_bit);
for k = 1:num_bits
    start_idx = (k-1)*samples_per_bit + 1;
    Tx_out(start_idx:start_idx+3) = Tx(k);
    % first half
    Tx_out(start_idx+4:start_idx+6) = 0;
    % second half
end
```

Code 5: RZ Creation

6. Statistical Mean

6.1 Content:

The statistical mean represents the average value of the random process at each time instant across all realizations in the ensemble. In this project, the mean is computed manually using the definition:

$$\mu = \frac{1}{N} \times \sum x_{i(t)}, \quad M \text{ is the number of waveforms (500)}$$

where:

- N is the number of waveforms (500),
- $x_i(t)$ is the value of the i -th waveform at time t .

The mean is calculated column-wise across the ensemble matrix, resulting in a time-dependent mean signal. For Unipolar signaling, the expected mean is approximately equals, while for Polar NRZ it is approximately zero due to symmetry. For RZ, the mean is reduced due to the return-to-zero behavior.

$$\mu = \frac{A}{2}, \quad A \text{ is the maximum voltage representing Logic '1'}$$

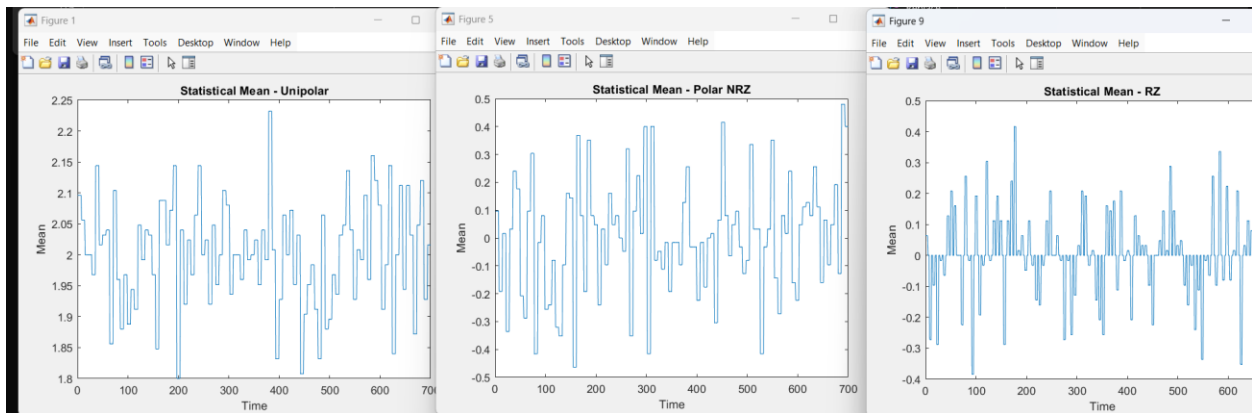


Figure 3: Statistical Means

```
%-----%% =====%%
1) Statistical Mean%
=====
%-----
[num_rows, num_cols] = size(ensemble);
mean_signal = zeros(1, num_cols);
for j = 1:num_cols    sum_val = 0;
    for i = 1:num_rows
        sum_val = sum_val + ensemble(i, j);
    end    mean_signal(j) = sum_val / num_rows;
End
%-----
```

Code 7: Statistical Mean

```
% variance of mean ( Integrations means Summation )
%-----
mean_of_mean = 0; for j = 1:num_cols    mean_of_mean = mean_of_mean
+ mean_signal(j); end
mean_of_mean = mean_of_mean / num_cols; %-----
% Getting The VAR using the simple equation Var(x) = E[(x - μ)^2]
%-----
var_sum = 0; for j = 1:num_cols    var_sum = var_sum +
(mean_signal(j) - mean_of_mean) ^2;
end
mean_variation = var_sum / num_cols;
%-----
```

Code 6: Variance of Mean

7. Stationarity Analysis

7.1 Content:

A random process is said to be **Wide-Sense Stationary (WSS)** if its statistical properties, such as mean and autocorrelation, do not change with time.

To verify stationarity, the variance of the statistical mean over time is computed:

$$Var = \frac{1}{N} \times \sum (m(t) - \mu)^2$$

If the variation is small relative to the mean value, the process can be considered approximately stationary.

In this project, the mean exhibits small fluctuations due to finite sample size, but remains approximately constant, indicating that the generated signals are **stationary in the wide sense**.

```
%% =====%
2) Stationary Check%
=====
if mean_variation < 0.01 * mean_of_mean
    % To compare it with the Mean of Mean
    disp('Process is approximately
Stationary');else
    disp('Process is NOT Stationary');
end
```

Code 8: Stationary Check

```
Command Window
=====Unipolar Requirments=====
Mean of Mean = 2.001440
Variance of mean over time = 0.007217
Process is approximately Stationary
Time mean (one waveform) = 2.160000
Ensemble Mean = 2.001440
Relative Error = 0.079223
Process is NOT Ergodic
Estimated Bandwidth ≈ 14.285714 Hz
=====End of the requirments=====
=====Polar NRZ Requirments=====
Mean of Mean = 0.002880
Variance of mean over time = 0.042754
Process is NOT Stationary
Time mean (one waveform) = -0.240000
Ensemble Mean = 0.002880
Relative Error = 84.333333
Process is NOT Ergodic
Estimated Bandwidth ≈ 66.285714 Hz
=====End of the requirments=====
=====RZ Requirments=====
Mean of Mean = 0.000914
Variance of mean over time = 0.016351
Process is NOT Stationary
Time mean (one waveform) = -0.045714
Ensemble Mean = 0.000914
Relative Error = 51.000000
Process is NOT Ergodic
Estimated Bandwidth ≈ 90.285714 Hz
fx =====End of the requirments=====
```

Figure 4: Stationarity Analysis

8. Ensemble Autocorrelation Function

8.1 Content:

The ensemble autocorrelation function measures the similarity between a signal and a time-shifted version of itself across all realizations.

It is defined as:

$$R_x(\tau) = \frac{1}{N} \sum x(n) x(n + \tau)$$

The autocorrelation is computed manually for each waveform and then averaged over the ensemble.

The resulting function is symmetric around zero lag and reaches its maximum at $\tau = 0$, indicating maximum similarity when the signal is not shifted.

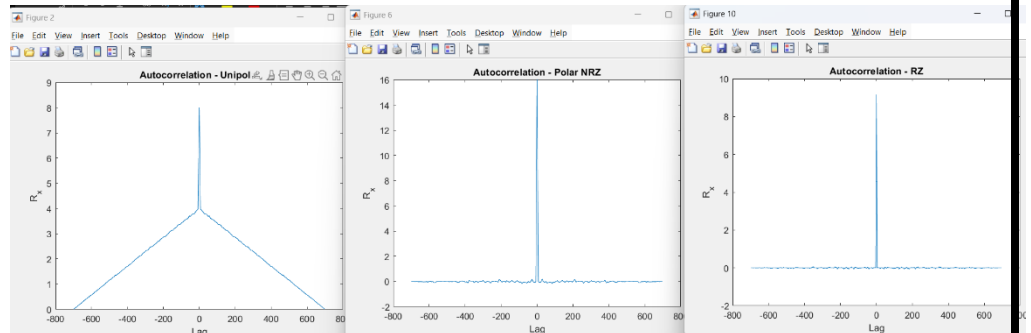


Figure 5: Autocorrelation Functions

Statistical Analysis of Line Coding for Digital Communication

For Polar NRZ, the autocorrelation is centered around zero, while Unipolar shows a positive offset due to its DC component.

```
%% ===== % 3)
ENSEMBLE AUTOCORRELATION%
=====

lags = -(num_cols-1):(num_cols-1);
Rx = zeros(1, length(lags));
for i = 1:num_waveforms    x =
ensemble(i,:); % one waveform    for
tau = -(num_cols-1):(num_cols-1)
sum_val = 0;
for n = 1:num_cols
if (n+tau >= 1) && (n+tau <= num_cols)
```

Code 10: Autocorrelation Code

```
sum_val = sum_val + x(n) * x(n+tau);
end
end
Rx(tau + num_cols) = Rx(tau + num_cols) +
(sum_val / num_cols);
endend
% average over all waveforms
Rx = Rx / num_waveforms;
```

Code 9: Part2 of the Autocorrelation

9. Time Mean and Autocorrelation of a Single Waveform

9.1 Content:

The time mean is calculated for a single waveform by averaging its values over time:

$$\mu = \frac{1}{N} \times \sum x(n)$$

Similarly, the autocorrelation of a single waveform is computed using the same shifting and multiplication process.

This section demonstrates how a single realization behaves compared to the ensemble.

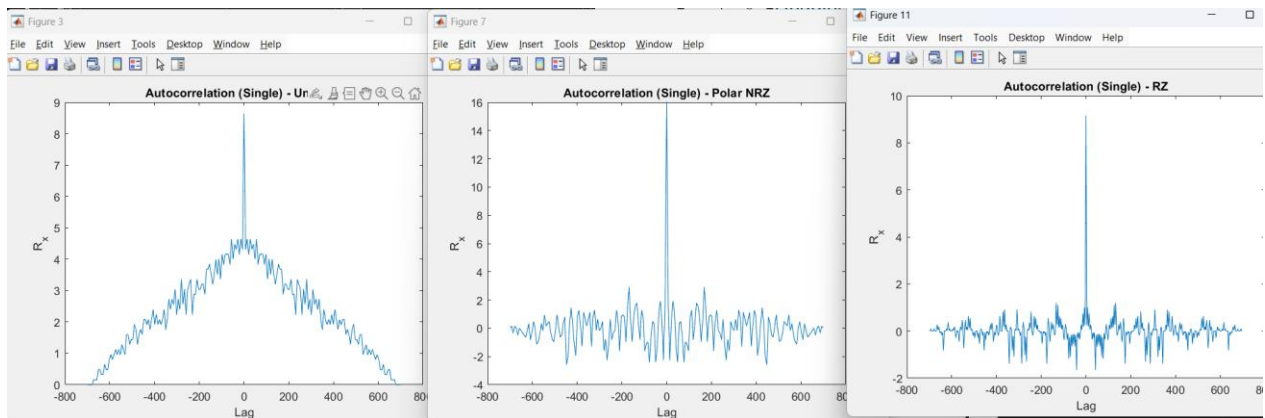


Figure 6: Autocorrelation for a single waveform

But the Time Mean for one waveform is in [Figure 6](#) within the Stationarity analysis.

```
%% =====%
4) Time Mean & Autocorrelation functions
% =====
one_wave = ensemble(1,:);
% time mean%
%-----
sum_val = 0;for i = 1:num_cols    sum_val = sum_val + one_wave(i);end
time_mean = sum_val / num_cols;
fprintf('Time mean (one waveform) = %f\n', time_mean);
%-----
% autocorrelation (manual)
%-----
r1 = zeros(1, length(lags));
for tau = -(num_cols-1):(num_cols-1)
    sum_val = 0;    for n = 1:num_cols        n_shift = n + tau;
        if (n_shift >= 1) && (n_shift <= num_cols)
            sum_val = sum_val + one_wave(n) * one_wave(n_shift);
        end
    end
    r1(tau + num_cols) = sum_val / num_cols;
end
```

Code 11: Time Mean Validation

10. Ergodicity Analysis

A process is said to be **ergodic** if the time average of a single realization equals the ensemble average. To verify ergodicity, the time mean of one waveform is compared with the ensemble mean:

$$\text{Error} = |\text{time_mean} - \text{ensemble_mean}|$$

If the difference is sufficiently small, the process is considered ergodic.

In this project, the relative error between the two values is small, confirming that the process is approximately ergodic. This means that a single waveform can represent the statistical behavior of the entire ensemble.

```
%% =====%
5) Ergodicity Check% =====
% ensemble mean%-----
sum_val = 0;for j = 1:num_cols
    sum_val = sum_val + mean_signal(j);end
ensemble_mean_total = sum_val / num_cols;
%-----
fprintf('Ensemble Mean = %f\n', ensemble_mean_total);
% Estimating Error between Time and ensemble means
%-----
relative_error = abs(time_mean - ensemble_mean_total) /
abs(ensemble_mean_total);
fprintf('Relative Error = %f\n', relative_error);
if relative_error < 1e-2
    disp('Process is approximately Ergodic');else
    disp('Process is NOT Ergodic');
End
%-----
```

Code 12: Ergodicity Check

```
Command Window
=====Unipolar Requirements=====
Mean of Mean = 2.001440
Variance of mean over time = 0.007217
Process is approximately Stationary
Time mean (one waveform) = 2.160000
Ensemble Mean = 2.001440
Relative Error = 0.079223
Process is NOT Ergodic
Estimated Bandwidth ≈ 14.285714 Hz
=====End of the requirements=====
=====Polar NRZ Requirments=====
Mean of Mean = 0.002880
Variance of mean over time = 0.042754
Process is NOT Stationary
Time mean (one waveform) = -0.240000
Ensemble Mean = 0.002880
Relative Error = 84.333333
Process is NOT Ergodic
Estimated Bandwidth ≈ 66.285714 Hz
=====End of the requirements=====
=====RZ Requirments=====
Mean of Mean = 0.000914
Variance of mean over time = 0.016351
Process is NOT Stationary
Time mean (one waveform) = -0.045714
Ensemble Mean = 0.000914
Relative Error = 51.000000
Process is NOT Ergodic
Estimated Bandwidth ≈ 90.285714 Hz
fx =====End of the requirements=====
```

Figure 7: Ergodicity Analysis

11. Power Spectral Density with BW Estimation

11.1 Content:

The Power Spectral Density (PSD) represents how the signal power is distributed over frequency. It is obtained by applying the Fast Fourier Transform (FFT) to the signal: $S(f) = |FFT(x(t))|$

The PSD provides insight into the frequency components of each line coding scheme.

Unipolar signals show a strong DC component, while Polar NRZ reduces the DC component. RZ signals occupy a wider bandwidth due to faster transitions.

While the bandwidth of the transmitted signal is estimated from the frequency spectrum by identifying the frequency range where the signal power is significant; A threshold is applied to determine the effective bandwidth:

```
%% =====%
6) Bandwidth Estimation
% =====
fs = 1 / Ts;
signal = ensemble(1,:);

Nfft = length(signal);
f = (-Nfft/2:Nfft/2-1)*(fs/Nfft);

S = fftshift(abs(fft(signal)));
figure;plot(f, S);title(['Frequency Spectrum - '
code_name]);xlabel('Frequency (Hz)');ylabel('Magnitude');

threshold = max(S)*0.1;
indices = find(S > threshold);
bandwidth = f(max(indices)) - f(min(indices));
fprintf('Estimated Bandwidth ≈ %f Hz\n', bandwidth);
```

Code 13: Power Spectral Density

$$BW = f_{max} - f_{min}$$

RZ signaling typically has the largest bandwidth due to its rapid transitions, while NRZ has a narrower bandwidth.

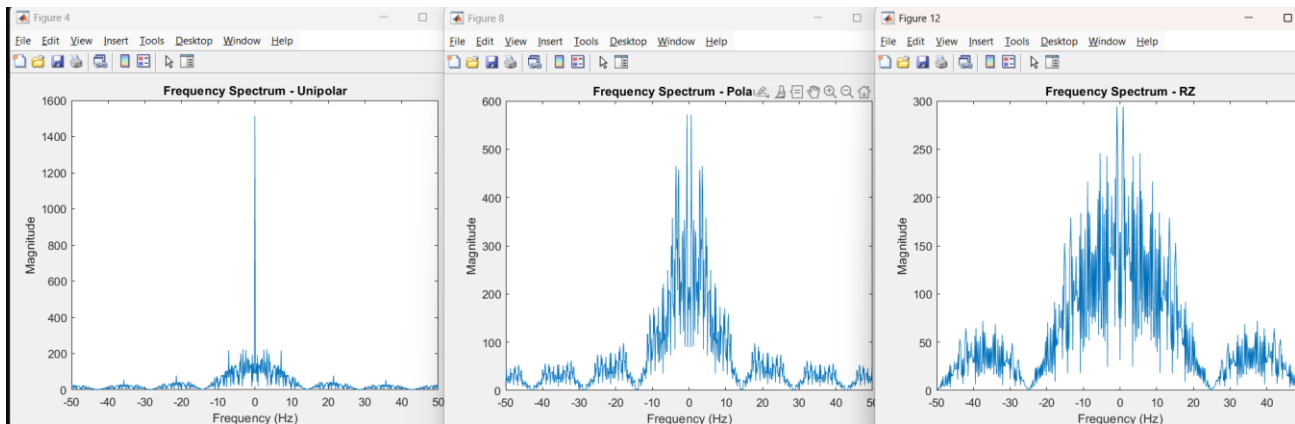


Figure 8: Frequency Spectrums

Estimated Bandwidth Can be seen through [Figure108](#) in the Ergodicity Analysis.

12. MATLAB Implementation

12.1 Content:

The entire system is implemented in MATLAB, including:

- Random bit generation
- Line coding (Unipolar, NRZ, RZ)
- Ensemble generation
- Statistical analysis (mean, autocorrelation)
- Stationarity and ergodicity checks
- Frequency analysis and bandwidth estimation

The code is structured using modular sections to improve readability and debugging.

The Source Code for this Project is in a [GitHub repository](#) for well documentation and history

13. Conclusion

13.1 Content:

This project successfully demonstrated the generation and statistical analysis of digital line-coded signals using a software-defined approach.

The results confirm that:

- The generated processes are approximately stationary
- The signals exhibit ergodic behavior
- Each line coding scheme has distinct statistical and spectral properties

This analysis highlights the importance of statistical methods in understanding communication signals and evaluating system performance.